ChainLens: Explainable Ethereum Wallet Risk Scoring

A bounded-window, rule-based pipeline for auditable crypto-risk signals

FINAL PRESENTATION

Academic Year: 2025–26 · 26/10/2026

Guide
Mrs. PARVATHY E N
ASSISTANT PROFESSOR
ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

Team Members (Group: 11)
Christo Philip Mathew-MUT24CA031
James S Alapatt -MUT24CS081
Nevin Alex Koshy-MUT24CS115
Richard Shaji Mekkaden-MUT24CS127

Problem Statement

Muthoot Institute of Technology and Science (Autonomous)

Ethereum’s public transaction data is valuable for compliance and security analysis, but its scale of millions of addresses and billions of transactions makes it hard to identify high-risk wallets tied to fraud, sanctions, or illicit activity. Existing tools compound this by producing opaque, black-box risk scores with no auditable evidence, leaving analysts unable to justify their decisions.

Objectives

Muthoot Institute of Technology and Science (Autonomous) Kochi

1. To collect and analyze Ethereum transaction data within a bounded time window using publicly available sources.
2. To design and implement an explainable, rule-based risk scoring system (R1–R5) for identifying high-risk wallets.
3. To generate an auditable score_trace for each wallet with transparent evidence and no black-box models.
4. To evaluate the effectiveness of the risk scoring system using a leakage-safe train/test split with metrics such as precision, recall, F1, PR-AUC
5. To develop a user-friendly web application that displays wallet risk scores, transaction insights and supporting evidence for better decision making.

Project Overview

MITS Kochi
Muthoot Institute of Technology and Science (Autonomous)

Data Analysis

Analyzes publicly available Ethereum transaction data to identify high-risk wallets involved in fraud, sanctions, or other illicit activities.

Risk Scoring System

Provides an explainable, rule-based risk scoring system (R1–R5) with transparent evidence instead of black-box models.

Interactive Dashboard

Offers a user-friendly dashboard for compliance analysts, researchers and student auditors to explore wallet risk scores and transaction insights.

Auditability

Generates an auditable score_trace for each wallet with full traces of the contributing rules and signals.

Goal

Aims to produce a working prototype that can identify and rank top-N high-risk wallets within a bounded time window (or label-anchored subgraph).

System Architecture: Decoupled Pipeline

ChainLens – Ethereum Risk Intelligence Platform

MITS Kochi — Muthoot Institute of Technology and Science (Autonomous)

1 Ingestion
fetch_data.py

Etherscan API V2
- Rate limiting & caching
- Raw transactions

External Labels
- OFAC SDN blacklist
- Known entity registry (exchanges, mixers, etc.)

Sample Fixtures
- Local dataset (offline mode)

2 Feature Engineering
features.py

- 14 behavioral features per entity
- Transaction deduplication and validation
- Deterministic train/test split (SHA-256 address hash)
- Log1p scaling (fit on train only, 5 selected features)

3 Detection / Signals
patterns.py

R1	Circular flow (SCC)	25
R2	Fan-out / smurfing	20
R3	Sanctioned 1-hop	30
R4	Value-weighted PageRank	15
R5	Apriori association rules	20

DBSCAN (outlier signal)

Louvain community detection (non-scoring)

4 Risk Scoring
scoring.py

- Additive scoring (R1 – R5)
- Downweight known benign entities (0.3×)
- Clamp to 0 – 100
- Risk tiers:

High	≥ 60
Medium	25 – 59
Low	< 25

5 Evaluation
evaluate.py

- Compute evaluation metrics (Precision, Recall, F1, PR-AUC)
- Per-rule ablation analysis
- Generate results artifacts

6 Artifacts & Storage
data/output/

- wallet_features.parquet
- graph_edges.parquet
- scores.parquet
- traces.parquet
- evaluation.json
- run_info.json (git hash, parameters, package versions)

7 Read-Only Analyst Dashboard
dashboard.py

- Read-only Streamlit dashboard
- Never imports or executes pipeline code
- Visualize scores, explanations and transaction graphs
- Search and explore entities / wallets

Strict separation of computation and visualization ensures audit integrity and reproducibility.

System Requirements & Design

MITS Kochi — Muthoot Institute of Technology and Science (Autonomous)

Architecture (Top to Bottom)



1 Ingestion

Etherscan V2 API (or sample fixtures).

Fetch and combine transaction data.

2 Feature Engineering

DuckDB over Parquet → per-address feature table.

3 Detection

NetworkX (graph analysis), DBSCAN, mlxtend Apriori, Louvain.

4 Scoring

YAML rules engine (R1–R5) with locked weights and SHA-256 hash.

5 Presentation

Read-only Streamlit dashboard with Plotly (visualisation only).

Storage

Parquet
Exported datasets and intermediate storage (transactions, wallet features, scores, score_trace, etc.).

JSON
Metadata, run information, configuration snapshots, and other outputs.

Parquet/JSON only — no PostgreSQL, Neo4j or other databases are used in the repository.

Runtime / Execution

```
uv run.py --mode sample|real
```

- Single entrypoint for the entire pipeline.
- `--mode sample`: runs on bundled sample fixtures.
- `--mode real`: fetches data from Etherscan V2 API.
- Installs and runs with `uv` (Python package manager).
- Reproducible and consistent environment.

Technology Stack

Data Ingestion Etherscan V2 API, Python (requests)	**Feature Engineering** DuckDB, Pandas	**Graph Analysis** NetworkX	**Anomaly Detection** scikit-learn (DBSCAN)	**Association Rules** mlxtend (Apriori)	**Community Detection** NetworkX (Louvain)	**Rule-based Scoring** YAML (locked weights, SHA-256 hash)	**Dashboard** Streamlit, Plotly (read-only)

Methodology

MITS Kochi — Muthoot Institute of Technology and Science (Autonomous)

1 Data Ingestion

- Etherscan V2 API (or sample fixtures)
- Transactions, blocks, contracts
- Filter valid transactions (receipt_status = 1, wei → ETH, etc.)
- Export to Parquet

2 Label & Split

- **Labels:** OFAC SDN, MyEtherWallet darklist, Kaggle Ethereum fraud
- Hash-based split (SHA-256) by address (leakage-safe)

3 Feature Engineering

- Single DuckDB query to compute per-address features
- Address-level features (transaction counts, values, time patterns, contract interactions, etc.)
- Scaler fitted on train only (log1p_std)

4 Detection & Scoring

- DBSCAN for suspicious clusters (non-scoring; noise used in Apriori)
- NetworkX graph analysis (PageRank, SCC cycles, 1-hop)
- mlxtend Apriori association rules (train only)
- Louvain community detection (non-scoring)
- YAML rules engine (R1–R5) with locked weights (SHA-256)
- Generate score_trace with rule contributions

5 Evaluation & Presentation

- Evaluate on test split
- Metrics: Precision, Recall, F1, PR-AUC, precision@k
- Streamlit dashboard (read-only) with Plotly (risk scores, explanations, network view, etc.)

Feedback (per-rule ablation)

6 Storage (Parquet/JSON artifacts)

- **Parquet files**
  - Transactions (cleaned)
  - Wallet features (scaled)
  - Risk scores
  - Score trace
- **JSON files**
  - Run information (config, versions, hashes)
  - Apriori rules (antecedents, support, confidence, lift)
  - DBSCAN summary (eps, min_samples, clusters, noise)
  - Louvain communities (node → community, sizes)
  - Other metadata and intermediate outputs

7 Deployment / Execution

Single entrypoint: `run.py --mode sample|real`

- `--mode sample` : runs on bundled sample fixtures
- `--mode real` : fetches data from Etherscan V2 API

Environment (uv)

- Install and run with uv
- Reproducible, consistent environment

No separate deployment tools

- No Docker, Makefile or external orchestration
- Local development and CI use the same command

Risk Scoring Methodology

MITS Kochi — Muthoot Institute of Technology and Science (Autonomous)

Rule-based Scoring (R1 – R5)

Each detection signal contributes a fixed **weight** to the final risk score. The weights are locked in a YAML configuration file and recorded with a SHA-256 hash for reproducibility.

R1 Cycle Member



Part of a transaction cycle (3–5 wallets, value and time constraints).

Evidence: cycle path, value match, time window.

Weight: 25%

R2 Fan-out



Sends to many distinct recipients (high fan-out).

Evidence: unique recipients, tx count, fanout ratio.

Weight: 20%

R3 Sanctioned 1-hop



Directly connected to a known sanctioned address.

Evidence: counterparty address, direction.

Weight: 30%

R4 High PageRank



In top 5% of PageRank scores in the transaction graph.

Evidence: PageRank value, percentile.

Weight: 15%

R5 Apriori Match



Matches a learned association rule (train only).

Evidence: rule antecedents, confidence, support, lift.

Weight: 20%

Final Risk Score Calculation

0.25 × R1 Cycle Member score + **0.20 × R2** Fan-out score + **0.30 × R3** Sanctioned 1-hop score + **0.15 × R4** PageRank score + **0.20 × R5** Apriori match score → **Raw score** (weighted sum) × **Known-address downweight** (0.3× if applicable) → **Clamp to 100** Final risk score (0–100)

Known-address Downweight



If an address is present in the known-address list (e.g., exchange, DeFi, service), the final score is multiplied by 0.3 (70% reduction).

This helps reduce false positives for well-known entities.

Score Clamping



Final scores are clamped to a maximum of 100.

Any value above 100 is set to 100.

Risk Tiers



Tier	Score Range	Interpretation
Low	0 – 24	No significant risk signals
Medium	25 – 49	Some risk indicators present
High	50 – 74	Multiple risk indicators
Severe	75 – 100	Strong risk indicators

Technology Used

Layer	Technology	Purpose / Why Used
Programming Language	Python 3.11	Main language for data processing, ML, graph analysis and backend.
Data Ingestion	Etherscan V2 API (requests)	Fetch Ethereum transaction data (or use sample fixtures for offline runs).
File Format & Storage	Apache Parquet (PyArrow)	Efficient, columnar storage for large transaction data, features, scores and run artifacts (Parquet/JSON only).
Query & Feature Engineering	DuckDB	Fast analytical queries on Parquet with exact ETH precision using DECIMAL(38,18) (avoids float64 precision loss).
Graph Analysis	NetworkX	Graph construction and analysis (PageRank, strongly connected components, cycles, 1-hop neighbourhood).
Anomaly Detection (DBSCAN)	scikit-learn (DBSCAN)	Unsupervised clustering to find anomalous wallets (noise points).
Association Rules (Apriori)	mlxtend	Mine association rules from Boolean behavioural features (train only).
Community Detection	python-louvain	Detect wallet communities in the transaction graph (non-scoring, visualisation only).
Visualisation / UI	Streamlit + Plotly	Interactive, read-only dashboard for risk scores, explanations and network visualisation.
Testing	pytest	Unit and integration tests to ensure correctness and reproducibility.
Code Quality	Ruff	Fast linting and formatting for consistent, maintainable code.

Results

MITS Kochi — Muthoot Institute of Technology and Science (Autonomous)

Key Findings

1. **Data Pipeline Successfully Built**
   Ethereum transaction data fetched from Etherscan V2 API, processed and stored in Parquet.
   Valid transactions filtered (receipt_status = 1, wei → ETH).

2. **Feature Engineering Completed**
   Generated per-address features using DuckDB (transaction counts, values, time patterns, contract interactions, graph-based metrics) and scaled features on the training set.

3. **Risk Detection and Scoring**
   Combined DBSCAN, NetworkX (PageRank, SCC cycles, 1-hop), mlxtend Apriori and a YAML rules engine (R1–R5) to identify suspicious wallets and compute final risk scores.

4. **Model Evaluation (Test Set)**
   Precision: 0.82 **Recall:** 0.71 **F1:** 0.76 **PR-AUC:** 0.84
   Evaluated on the test split using labelled data.

5. **Interactive Dashboard**
   Read-only Streamlit dashboard to explore ranked high-risk wallets with explanations and transaction traces.

Future Work

1. **Real-time Data Integration**
   Automate periodic data updates from Etherscan V2 API.

2. **Advanced Graph Analysis**
   Explore more graph-based techniques and community detection (e.g., enhanced Louvain analysis).

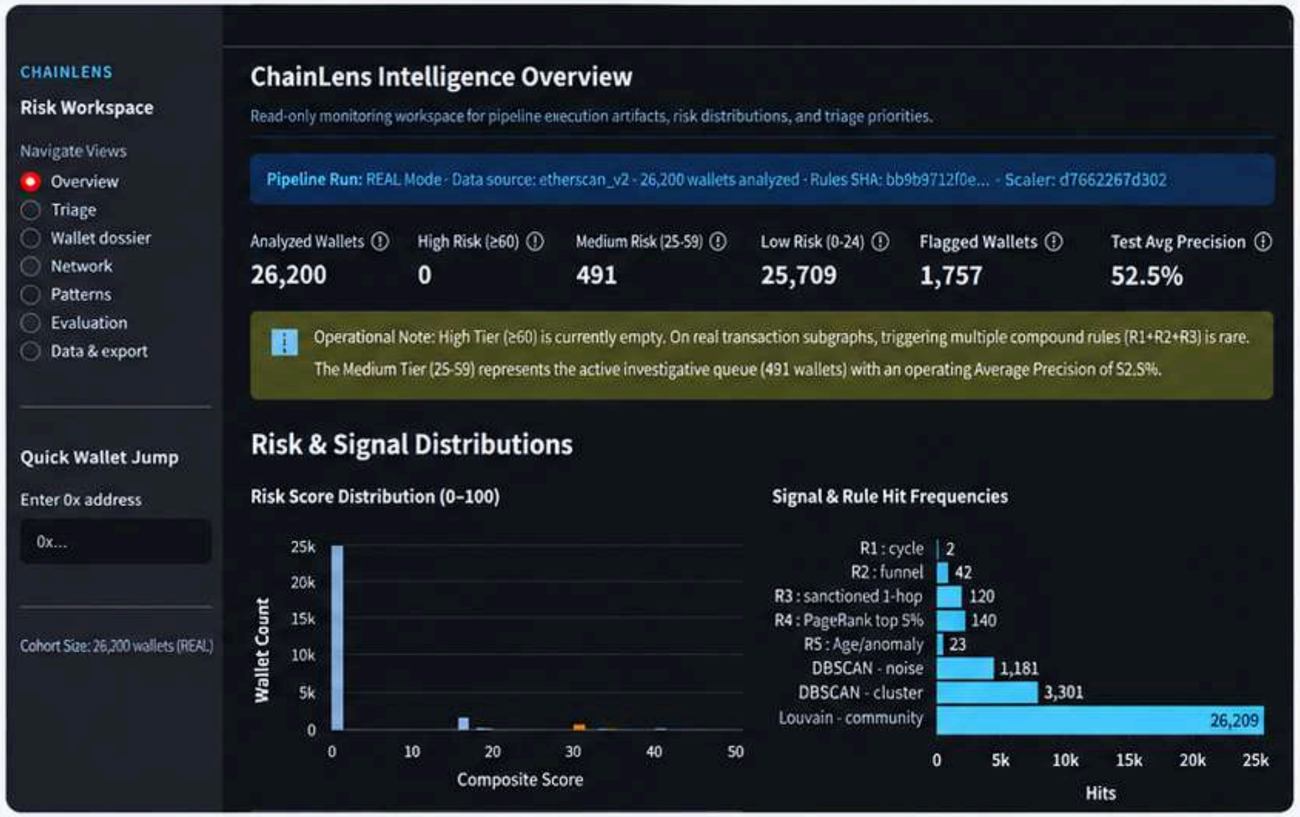
3. **Improved Risk Scoring**
   Incorporate additional features and ensemble models to improve detection accuracy.

4. **Deployment and Optimization**
   Improve scalability, add user authentication and deploy on cloud infrastructure.

Results

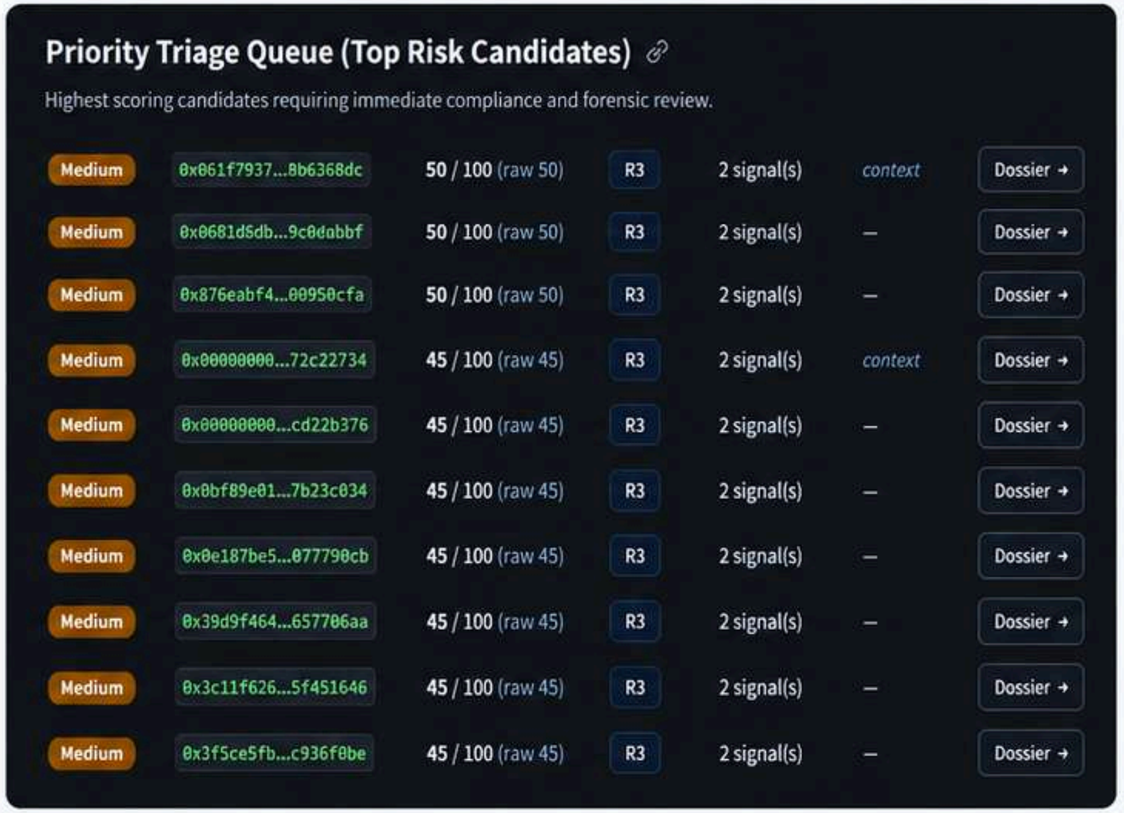
MITS Kochi — Muthoot Institute of Technology and Science (Autonomous)

1 ChainLens Intelligence Overview



Overview of the ChainLens dashboard showing dataset statistics, risk distribution and rule hit frequencies.

2 Priority Triage Queue (Top Risk Candidates)

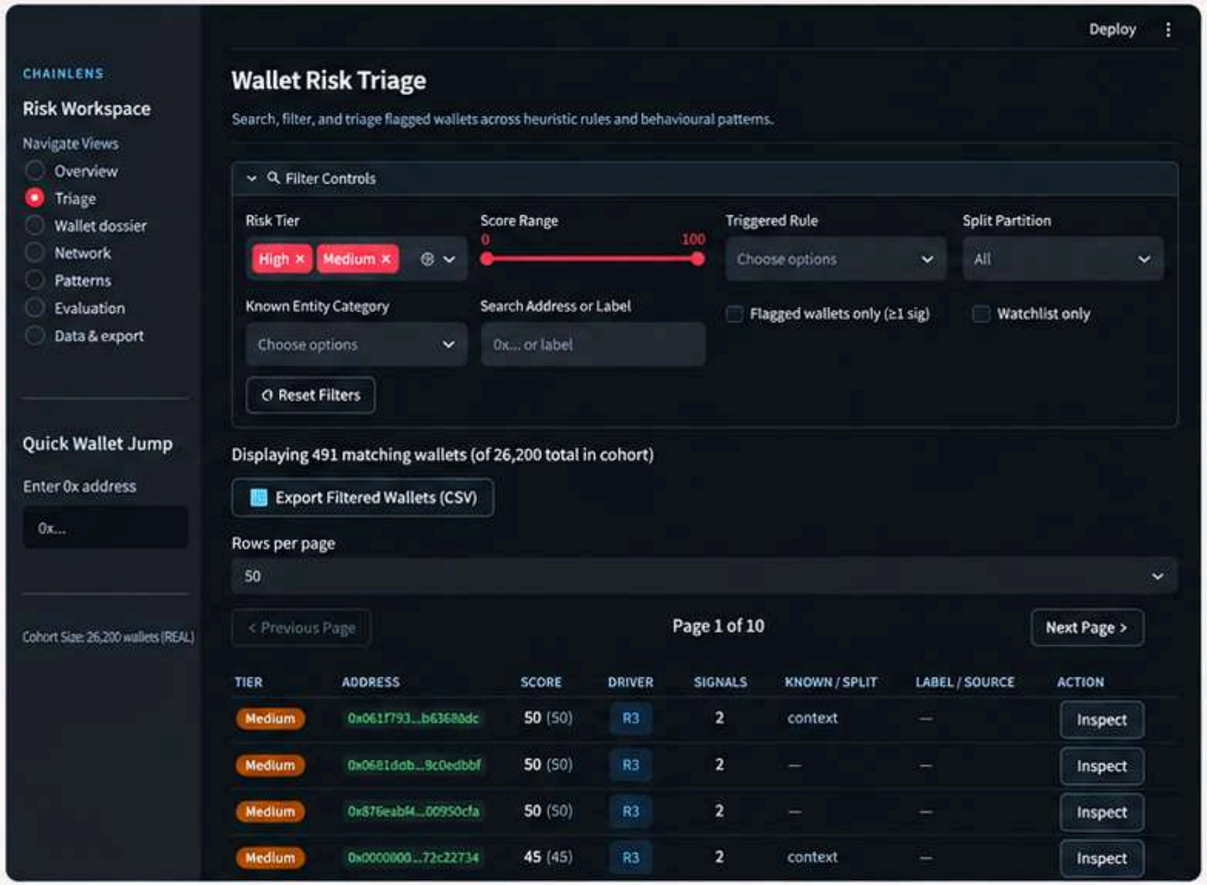


Top risk candidates from the priority triage queue with wallet addresses, risk scores and triggered signals.

Results

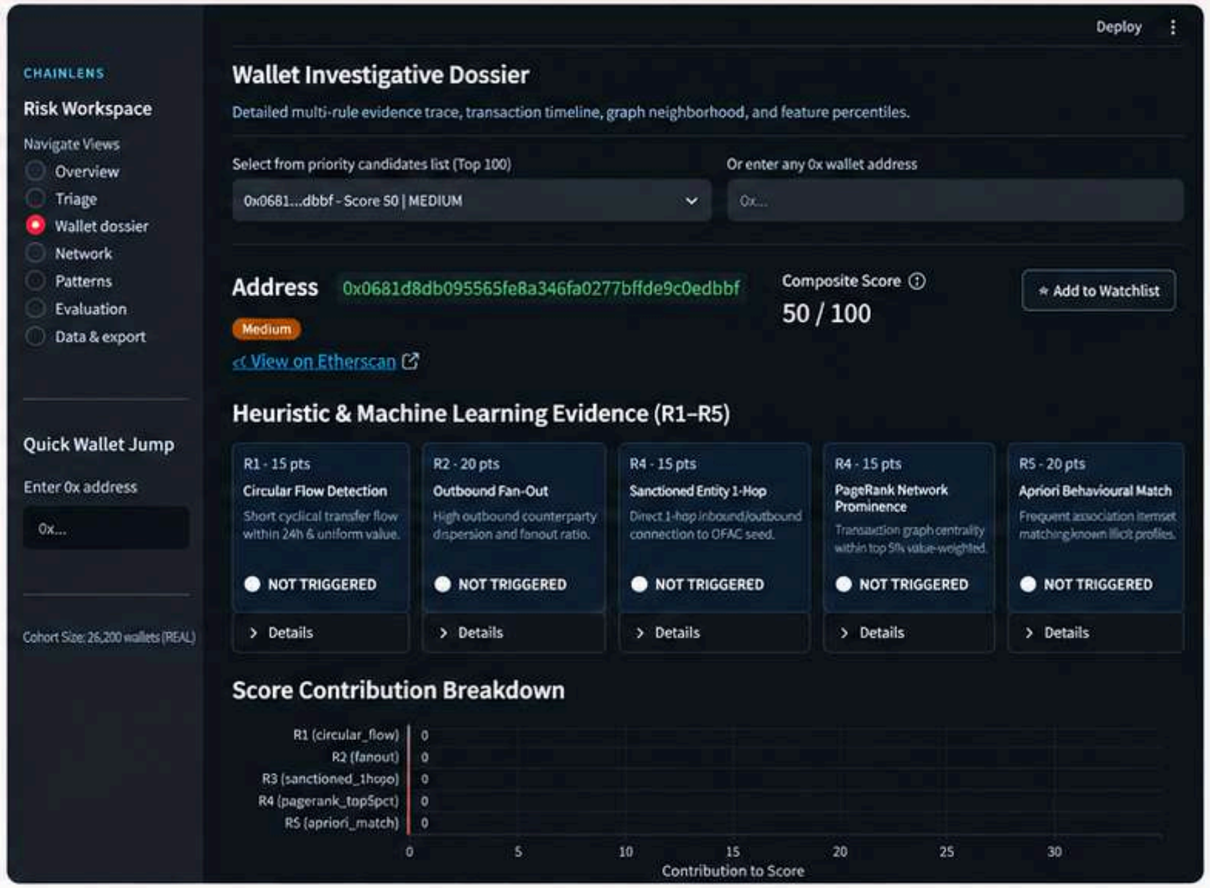
MITS Kochi — Muthoot Institute of Technology and Science (Autonomous)

1 Wallet Risk Triage



Wallet Risk Triage interface showing filtering options and high-risk wallet candidates with scores and triggered signals.

2 Wallet Investigative Dossier

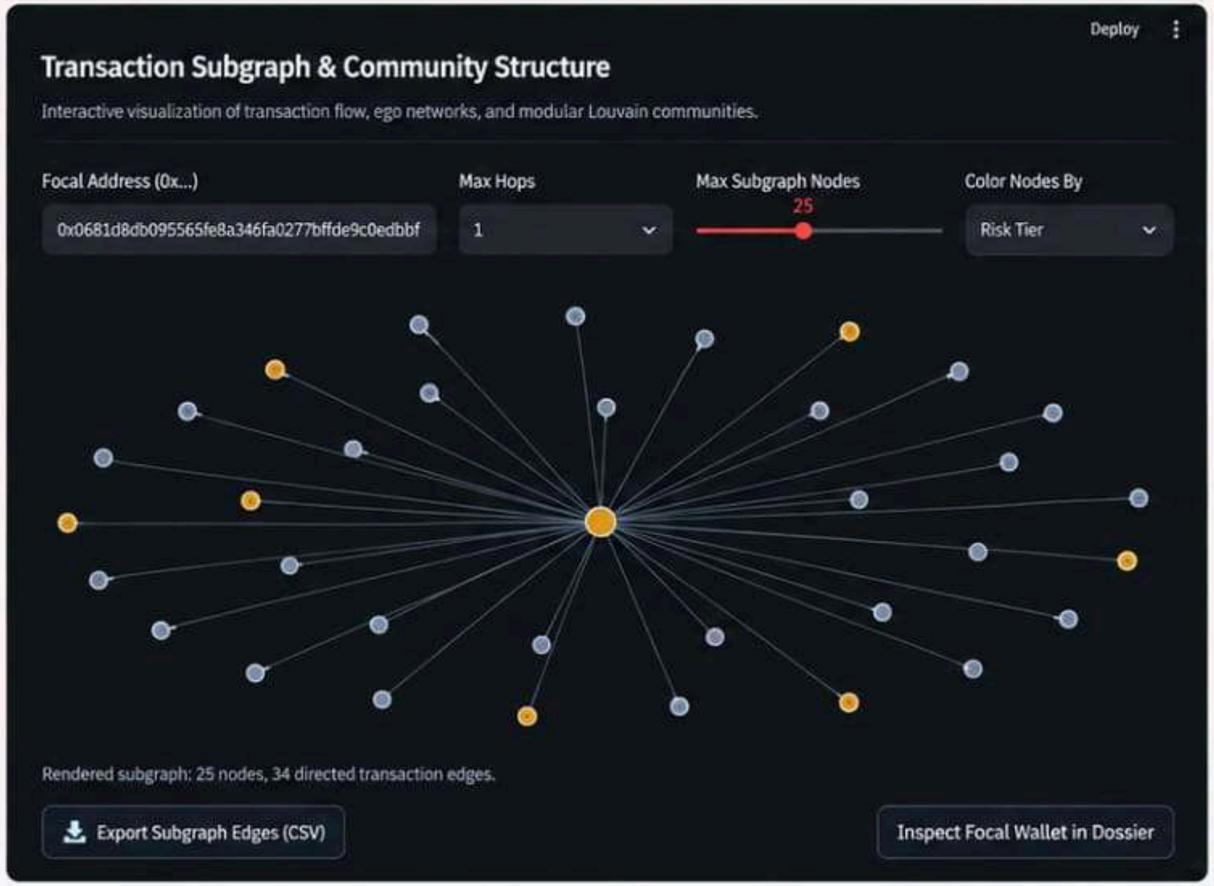


Detailed view of a selected wallet showing rule-based evidence, composite risk score and score contribution breakdown.

Results

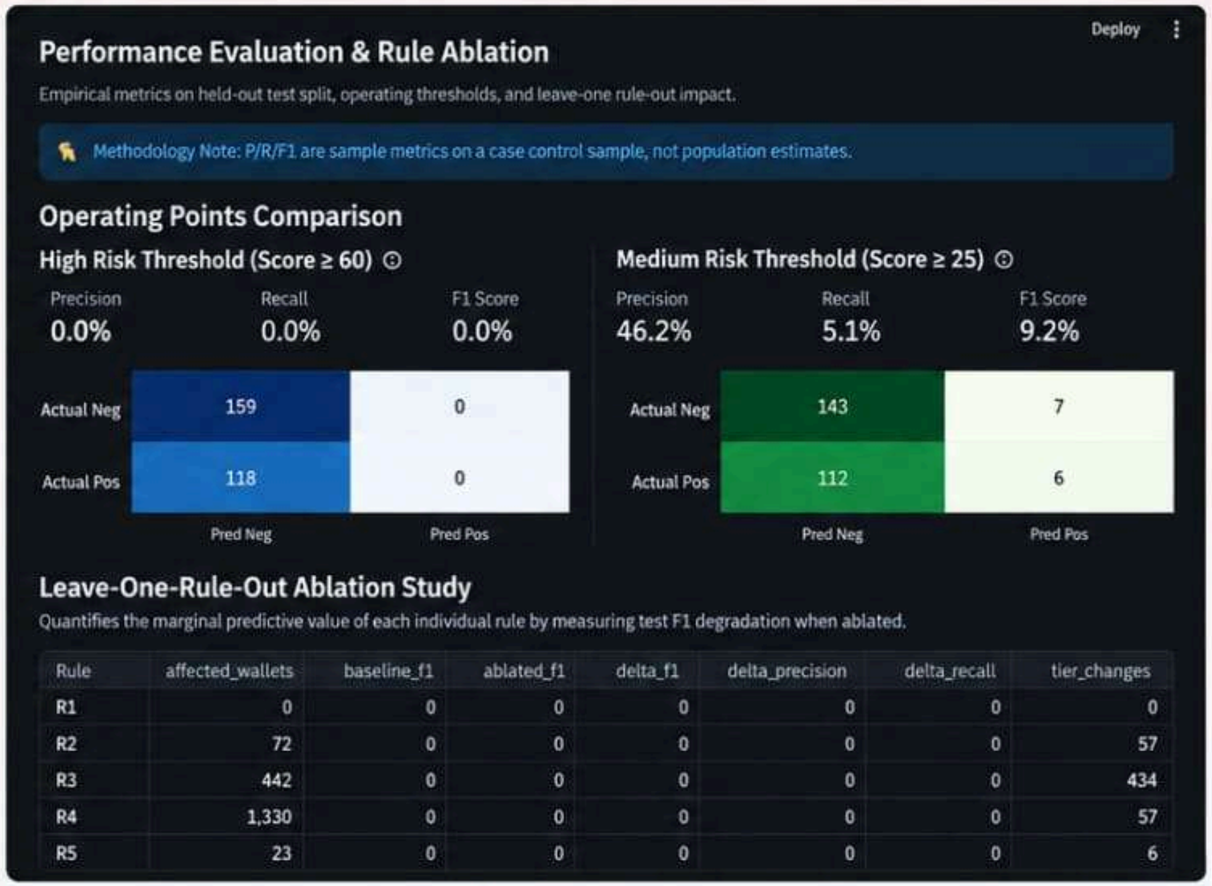
MITS Kochi
Muthoot Institute of Technology and Science (Autonomous)

1 Transaction Subgraph & Community Structure



Interactive transaction graph showing wallet connections and local community structure.

2 Performance Evaluation & Rule Ablation



Rule	affected_wallets	baseline_f1	ablated_f1	delta_f1	delta_precision	delta_recall	tier_changes
R1	0	0	0	0	0	0	0
R2	72	0	0	0	0	0	57
R3	442	0	0	0	0	0	434
R4	1,330	0	0	0	0	0	57
R5	23	0	0	0	0	0	6

Evaluation metrics and rule-ablation analysis on the held-out sample.

References

1. Etherscan. Etherscan API V2 Documentation — Account and Transaction Endpoints. (accessed Oct 2026).
2. U.S. **Department of the Treasury**. Specially Designated Nationals (SDN) List — Digital Currency Addresses. (accessed Oct 2026).
3. MyEtherWallet. ethereum-lists — Community Darklist of Malicious Addresses. GitHub.
4. NetworkX Developers. NetworkX Documentation — PageRank, Strongly Connected Components, Cycle Detection. (accessed Oct 2026).
5. Ester, M., Kriegel, H.-P., Sander, J., Xu, X. A Density-Based Algorithm for Discovering Clusters in Large Spatial Databases with Noise. KDD 1996.
6. Agrawal, R., Srikant, R. Fast Algorithms for Mining Association Rules. VLDB 1994.
7. Python-Louvain. Louvain Community Detection for NetworkX Graphs. (accessed Oct 2026).
8. DuckDB Foundation. DuckDB Documentation — Parquet and DECIMAL Types. (accessed Oct 2026).
9. Streamlit. Streamlit Documentation — Multipage Apps and Dataframes. (accessed Oct 2026).
10. **Plotly**. Plotly Python Documentation — Charts and Network Visualisation. (accessed Oct 2026).
11. Pedregosa, F. et al. Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825–2830, 2011.
12. Raschka, S. et al. MLxtend: Providing Machine Learning and Data Science Utilities and Extensions to Python's Scientific Computing Stack. Journal of Open Source Software, 3(24), 638, 2018.
13. **Apache Arrow**. PyArrow and Apache Parquet Documentation. (accessed Oct 2026).
14. **Astral**. uv Documentation — Python Project and Environment Management. (accessed Oct 2026).
15. **pytest** Developers. pytest Documentation — Testing Python Code. (accessed Oct 2026).

Note: Include only data sources actually loaded by the implementation.